

Problem Set 5

Hoa T. Vu

Neural Networks

Problem 1: Forward Pass.

The **SCReLU** (Squared Clipped ReLU) activation is defined as:

$$\text{SCReLU}(z) = \min(\max(z, 0), 1)^2$$

Consider the following two-hidden-layer network with input $\mathbf{x} = (1, 0, -1)^\top$:

Layer 1 (ReLU), $W^{(1)} \in \mathbb{R}^{4 \times 3}$, $\mathbf{b}^{(1)} \in \mathbb{R}^4$:

$$W^{(1)} = \begin{pmatrix} 1 & 2 & 0 \\ -1 & 1 & 1 \\ 0 & -1 & 2 \\ 1 & 0 & -1 \end{pmatrix}, \quad \mathbf{b}^{(1)} = \begin{pmatrix} 1 \\ 0 \\ -1 \\ 0 \end{pmatrix}$$

Layer 2 (SCReLU), $W^{(2)} \in \mathbb{R}^{3 \times 4}$, $\mathbf{b}^{(2)} \in \mathbb{R}^3$:

$$W^{(2)} = \begin{pmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \\ -1 & 0 & 1 & 1 \end{pmatrix}, \quad \mathbf{b}^{(2)} = \begin{pmatrix} 0 \\ 1 \\ -1 \end{pmatrix}$$

Output layer (softmax, 2 classes), $W^{(3)} \in \mathbb{R}^{2 \times 3}$, $\mathbf{b}^{(3)} \in \mathbb{R}^2$:

$$W^{(3)} = \begin{pmatrix} 1 & -1 & 2 \\ -1 & 1 & -2 \end{pmatrix}, \quad \mathbf{b}^{(3)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

- (a) Compute $\mathbf{z}^{(1)} = W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$ and $\mathbf{a}^{(1)} = \text{ReLU}(\mathbf{z}^{(1)})$.
- (b) Compute $\mathbf{z}^{(2)} = W^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}$ and $\mathbf{a}^{(2)} = \text{SCReLU}(\mathbf{z}^{(2)})$.
- (c) Compute $\mathbf{z}^{(3)} = W^{(3)}\mathbf{a}^{(2)} + \mathbf{b}^{(3)}$ and apply softmax:

$$\hat{y}_k = \frac{e^{z_k^{(3)}}}{e^{z_1^{(3)}} + e^{z_2^{(3)}}}, \quad k = 1, 2.$$

Which class does the network predict?

Problem 2: Forward Pass with Dropout.

Repeat Problem 1 with the following neurons **zeroed out** (dropped):

- Layer 1: neuron 1 (the first component of $\mathbf{a}^{(1)}$).
 - Layer 2: neuron 1 (the first component of $\mathbf{a}^{(2)}$).
- (a) Recompute $\mathbf{a}^{(1)}$ after applying dropout.
- (b) Recompute $\mathbf{a}^{(2)}$ after applying dropout.
- (c) Recompute the softmax output. Does the predicted class change?

Programming

Problem 3: Feedforward Network on Digits.

Use the following starter code to build a feedforward network that classifies handwritten digits (0–9).

```
import torch
import torch.nn as nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# Load data
X, y = load_digits(return_X_y=True)
X = X / 16.0 # normalize pixel values to [0, 1]

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42)

X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.long)
X_val = torch.tensor(X_val, dtype=torch.float32)
y_val = torch.tensor(y_val, dtype=torch.long)

# Fill in the model
model = nn.Sequential(
    # your layers here
    # input size: 64 (8x8 flattened image)
    # output size: 10 (one logit per digit class)
)

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

# training loop
train_losses, val_accs = [], []

for epoch in range(200):
    model.train()
```

```

logits = model(X_train)
loss    = loss_fn(logits, y_train)
optimizer.zero_grad()
loss.backward()
optimizer.step()
train_losses.append(loss.item())

model.eval()
with torch.no_grad():
    preds = model(X_val).argmax(dim=1)
    acc    = (preds == y_val).float().mean().item()
val_accs.append(acc)

print(f"Final validation accuracy: {val_accs[-1]:.3f}")

# --- plots ---
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))
ax1.plot(train_losses)
ax1.set_xlabel("Epoch"); ax1.set_ylabel("Loss")
ax1.set_title("Training Loss")
ax2.plot(val_accs)
ax2.set_xlabel("Epoch"); ax2.set_ylabel("Accuracy")
ax2.set_title("Validation Accuracy")
plt.tight_layout(); plt.show()

```

- (a) Fill in the model definition. Use at least one hidden layer with ReLU. Report your final validation accuracy.
- (b) Add `nn.Dropout` to your model. Does it improve validation accuracy? Report results with and without dropout.
- (c) Plot the training loss and validation accuracy curves. Describe what you observe: does the model converge? Does it overfit?

Problem 4: Convolutional Network on Digits.

The same digits dataset can be treated as 8×8 grayscale images. The starter code below trains a simple CNN. Your task is to modify the architecture and improve validation accuracy.

```

import torch
import torch.nn as nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# --- data ---
X, y = load_digits(return_X_y=True)
X = X.reshape(-1, 1, 8, 8) / 16.0      # shape: (n, 1, 8, 8)

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42)

```

```

X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.long)
X_val    = torch.tensor(X_val,    dtype=torch.float32)
y_val    = torch.tensor(y_val,    dtype=torch.long)

# --- baseline CNN ---
model = nn.Sequential(
    nn.Conv2d(1, 8, kernel_size=3, padding=1), # -> 8 x 8 x 8
    nn.ReLU(),
    nn.MaxPool2d(2),                          # -> 8 x 4 x 4
    nn.Flatten(),
    nn.Linear(8 * 4 * 4, 10),
)

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

train_losses, val_accs = [], []

for epoch in range(200):
    model.train()
    logits = model(X_train)
    loss = loss_fn(logits, y_train)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    train_losses.append(loss.item())

    model.eval()
    with torch.no_grad():
        preds = model(X_val).argmax(dim=1)
        acc = (preds == y_val).float().mean().item()
    val_accs.append(acc)

print(f"Baseline validation accuracy: {val_accs[-1]:.3f}")

```

- (a) Run the baseline and report validation accuracy.
- (b) Modify the CNN by adding a second convolutional layer (after the first `MaxPool2d`) with 16 output channels. Adjust the `nn.Linear` input size accordingly. Report the new validation accuracy.
- (c) Try changing the filter size (e.g. `kernel_size=5`) or adding a fully connected hidden layer before the output. Report what changes you made and whether they helped.
- (d) Compare the best CNN result to your best feedforward result from Problem 3. Which architecture performs better, and why do you think that is?